Cairo University
Faculty of Computing and Artificial Intelligence

Material Stream Identification System

SCS462 - Machine Learning

Team Members

Haneen Hisham — 20236032
Shaza Moatasem — 20236050
Ziad Tarek — 20236043
Seif Waleed — 20236046
Mohamed Ahmed — 20237011

1. Data Augmentation

1.1 Overview

The provided dataset contains images across six material classes with counts ranging from **110 to 476** images per class. A data augmentation pipeline was implemented to balance all classes to exactly **500 images each**, resulting in a **53% overall increase** from 1960 to 3000 images, exceeding the 30% minimum requirement.

1.2 Dataset Before and After Augmentation

Class	Original Count	Images Added	Final Count
Cardboard	259	241	500
Glass	401	99	500
Metal	328	172	500
Paper	476	24	500
Plastic	386	114	500
Trash	110	390	500

Trash required the most augmentation (390 new images) while paper required the least (24 new images), reflecting the original dataset imbalance.

1.3 Augmentation Techniques

Each augmentation is applied randomly and independently, meaning a single image may receive any combination of the following transforms. This stochastic approach ensures that every generated image is unique and adds genuine variety to the dataset.

Technique	Probability	Details
Rotation (90°)	50%	Randomly rotates by 0°, 90°, 180°, or 270°
Horizontal Flip	50%	Mirrors the image left to right
Vertical Flip	30%	Mirrors the image top to bottom
Color Jitter (Brightness & Contrast)	70%	Randomly scales both in range [0.8, 1.2]
Gaussian Noise	30%	Adds noise with std between 10 and 50
Affine Transform	60%	Random rotation ±30°, scale 0.8–1.2, translation ±10%

Justification per technique:

- Rotations and flips simulate the fact that waste items are photographed in arbitrary orientations. A glass bottle lying on its side should be classified the same as one standing upright.
- Color jitter simulates varying lighting conditions, which is a major source of real-world variation in a camera-based system.
- Gaussian noise mimics sensor noise in lower-quality cameras, improving robustness.
- Affine transformations simulate slight perspective changes, distance variation, and off-center placement of objects.

1.4 Train / Validation Split

After augmentation, the full 500-image set for each class is split into training and validation subsets using an 80/20 ratio. The split is performed with a fixed random seed (42) to ensure reproducibility across runs.

Split	Images per Class	Total Images
Training	400	2400
Validation	100	600

2. Feature Extraction

2.1 Overview

Raw images cannot be fed directly into classical ML classifiers like SVM or k-NN. They must first be converted into fixed-length numerical vectors. Eight complementary feature descriptors were chosen and combined — capturing shape, color, and texture from multiple angles — producing a **738-dimensional feature vector** per image.

2.2 Image Preprocessing

Before any descriptor is computed, every image is resized to a fixed size of 128×128 pixels. This ensures all feature vectors have identical length regardless of the original image dimensions. Resizing uses area interpolation (cv2.INTER_AREA), which averages pixel values when downscaling and produces cleaner results than nearest-neighbor or bilinear methods.

2.3 Feature Descriptors

Histogram of Oriented Gradients (HOG)

HOG captures the shape and edge structure of an image. The image is divided into small cells, and for each cell a histogram of gradient directions is computed. Adjacent cells are grouped into overlapping blocks whose histograms are contrast-normalized to improve robustness to lighting variation.

Parameter	Value
Orientations	9 (each bin covers 20°)
Pixels per cell	32×32
Cells per block	2×2
Block normalization	L2-Hys
Gamma correction	Enabled

Using 32×32 pixel cells produces coarser, more global edge patterns that better capture the overall shape structure of material categories — more effective than finer cells for distinguishing materials at the category level.

Color Histograms (BGR + HSV + LAB)

32-bin histograms are computed for each channel across three color spaces: BGR, HSV, and LAB. Using multiple color spaces captures complementary aspects of color — HSV separates hue from brightness, LAB separates luminance from color components, and BGR provides raw pixel-level color information.

Parameter	Value
Color spaces	BGR, HSV, LAB
Bins per channel	32
Total channels	9 (3 per space)
Normalization	Sum normalization per channel
Output dimensions	288

Different materials have characteristic color distributions across these spaces. Metal is silver/grey with low saturation, glass is often clear or tinted, paper tends toward white, and plastic comes in bright saturated colors.

Color Moments

Mean, standard deviation, and median are computed for each HSV channel, providing a compact but effective summary of the color distribution. These three statistics capture the central tendency, spread, and skewness of colors without the overhead of a full histogram.

Parameter	Value
Color space	HSV
Statistics	Mean, Standard Deviation, Median
Output dimensions	9

Local Binary Patterns (LBP)

LBP captures the surface texture of an image. For every pixel, the descriptor compares it with its circular neighbors and encodes the result as a binary number. The uniform variant retains only patterns with at most two 0→1 or 1→0 transitions, which correspond to meaningful micro-structures such as edges, corners, and flat areas.

Parameter	Value
Radius	3
Number of points	24
Method	Uniform
Output dimensions	26

Cardboard has a coarse fibrous texture producing many edge-type LBP codes. Plastic surfaces are smooth and dominated by flat-area codes. Metal has a fine-grained reflective texture, and glass produces specular highlights — each resulting in a distinctive LBP profile.

Gray-Level Co-occurrence Matrix (GLCM)

GLCM captures how often pairs of pixel values appear next to each other in the image. From this matrix six texture properties are extracted: contrast, dissimilarity, homogeneity, energy, correlation, and ASM. Using three distances and four angles provides a comprehensive multi-scale texture description.

Parameter	Value
Distances	1, 2, 4
Angles	0°, 45°, 90°, 135°
Properties	contrast, dissimilarity, homogeneity, energy, correlation, ASM
Output dimensions	72

Hu Moments

Hu Moments are seven shape descriptors that are invariant to rotation, scale, and translation. They are log-transformed to compress the large numerical range of their values. These moments capture global shape properties that complement the local texture descriptors.

Parameter	Value
Number of moments	7
Transformation	Log-transform: $-\text{sign}(h) \times \log10(\lvert h \rvert + \varepsilon)$
Output dimensions	7

Edge Features

Canny edge detection is applied to extract four edge-based statistics: edge pixel density, number of contours, mean contour area, and standard deviation of contour areas. These capture the structural complexity and edge distribution of the material surface.

Feature	Description
Edge density	Fraction of pixels classified as edge pixels
Contour count	Number of detected contours
Mean contour area	Average area of detected contours
Contour area std	Variability in contour sizes

Gabor Filters

Gabor filters are applied at four orientations (0°, 45°, 90°, 135°). For each filter the mean and standard deviation of the response are recorded. Gabor filters are sensitive to texture at specific orientations, helping distinguish materials with directional surface patterns such as cardboard fibers or metal grain.

Parameter	Value
Orientations	0°, 45°, 90°, 135°
Kernel size	21×21
Statistics per filter	Mean and standard deviation
Output dimensions	8

2.4 Combined Feature Vector

Descriptor	Dimensions
HOG (32×32 cells, 9 orientations)	—
Color Histograms (BGR + HSV + LAB, 32 bins)	288
Color Moments (HSV mean, std, median)	9
LBP (radius=3, 24 points, uniform)	26
GLCM (6 properties, 3 distances, 4 angles)	72
Hu Moments (log-transformed)	7
Edge Features (Canny density + contours)	4
Gabor Filters (4 orientations, mean + std)	8
Total	738

2.5 Feature Scaling

After extraction, all features are standardized using StandardScaler (zero mean, unit variance). This is necessary because the eight descriptors operate on very different numerical scales. Without scaling, descriptors with larger raw values would dominate the distance calculations in k-NN and the optimization in SVM.

The scaler is fitted exclusively on the training set and then applied to both training and validation sets. Fitting the scaler on validation data would constitute data leakage and produce misleadingly optimistic results. The fitted scaler is saved to features/scaler.pkl and must be loaded in the real-time application before passing any feature vector to the classifier.

3. SVM Classifier

3.1 Overview

A Support Vector Machine (SVM) classifier was trained on the extracted 738-dimensional feature vectors to classify images into one of six material categories. SVM was chosen because it is well-suited for high-dimensional feature spaces, it finds the maximum-margin decision boundary between classes, and it performs strongly even with a relatively small number of training samples.

3.2 Kernel Choice

The RBF (Radial Basis Function) kernel was selected after experimentation with alternative kernels. The RBF kernel maps the input features into an infinite-dimensional space, allowing the classifier to learn non-linear decision boundaries between material classes. This is necessary because the relationship between feature values and material categories is not linearly separable.

Parameter	Value	Reason
Kernel	RBF	Handles non-linear class boundaries
Gamma	scale	Automatically scales based on feature variance
Class weight	balanced	Compensates for any remaining class imbalance
Probability	True	Required for the confidence-based rejection mechanism

3.3 Hyperparameter Tuning

The regularization parameter C controls the trade-off between achieving a smooth decision boundary and correctly classifying all training points. A manual search was performed over C = 1, 5, 10, 50, 100, 200, 500. The model was trained and evaluated on the validation set for each value, and the best-performing C was selected.

3.4 Results

The best performing configuration was **C = 5** with an **RBF kernel**, achieving a validation accuracy of **78.50%**.

Class	Precision	Recall	F1-Score	Support
Glass	0.77	0.74	0.76	100
Paper	0.86	0.83	0.85	100
Cardboard	0.82	0.90	0.86	100
Plastic	0.85	0.77	0.81	100
Metal	0.72	0.73	0.73	100

Class	Precision	Recall	F1-Score	Support
Trash	0.70	0.74	0.72	100
Overall	0.79	0.79	0.79	600

Cardboard achieved the highest recall (0.90) and paper the highest precision (0.86), reflecting their distinctive structural and color profiles. Glass remains the hardest class, frequently confused with metal due to overlapping reflective surface characteristics.

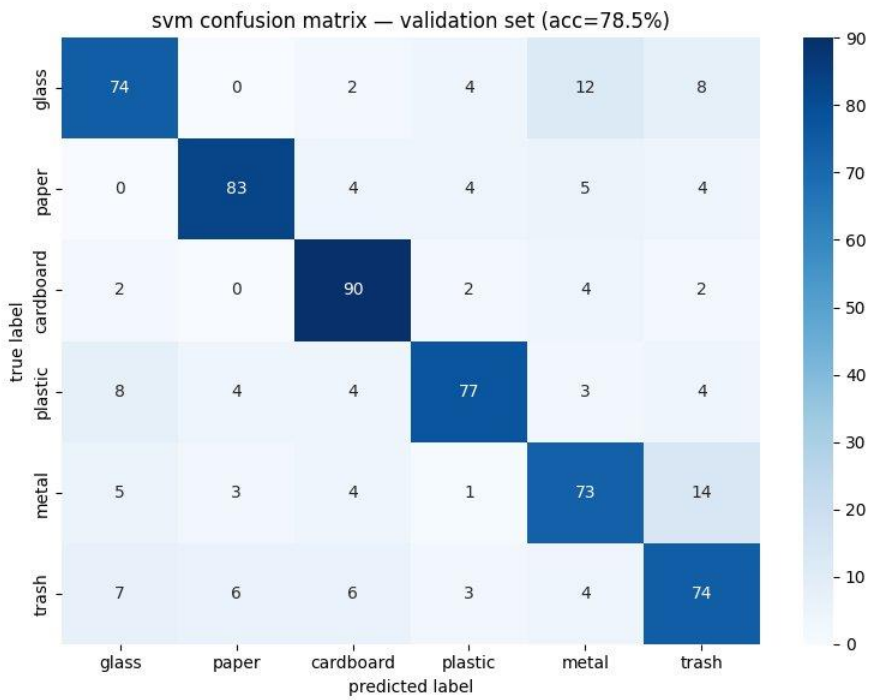

Figure 1: SVM Confusion Matrix — Validation Set (Accuracy = 78.5%)

3.5 Trained Model

The trained SVM model is saved as models/svm_model.pkl and is included in the repository.

4. k-NN Classifier

4.1 Overview

A k-Nearest Neighbors (k-NN) classifier was trained on the same 738-dimensional feature vectors as the SVM. k-NN classifies a new sample by finding the k closest training samples in feature space and taking a weighted vote among their labels. It was chosen as the second classifier because it makes no assumptions about the underlying data distribution, is straightforward to interpret, and provides a strong baseline for comparison against SVM.

4.2 Distance Metric

Manhattan distance was selected as the distance metric based on GridSearchCV results. Manhattan distance sums the absolute differences across all dimensions, making it less sensitive to outlier dimensions than Euclidean distance. This is particularly beneficial in high-dimensional feature spaces where a few large differences in individual dimensions can otherwise dominate the distance calculation.

4.3 Hyperparameter Search

Three hyperparameters were tuned simultaneously using GridSearchCV with 5-fold stratified cross-validation, optimizing for macro F1-score across 30 parameter combinations:

Hyperparameter	Values Tested
k (number of neighbors)	3, 5, 7, 9, 11
Weighting scheme	uniform, distance
Distance metric	euclidean, manhattan, minkowski

Stratified k-fold ensures each fold maintains the same class ratio as the full dataset. Optimizing for macro F1-score rather than accuracy pushes the model to perform well across all six classes equally, not just the easier ones. Distance weighting gives closer neighbors more influence (weight = 1/distance), which better handles noisy or overlapping class boundaries.

4.4 Results

The best performing configuration was **k = 7** with **manhattan distance** and **distance weighting**, achieving a validation accuracy of **66.50%** (macro F1: 0.668). The search covered 30 combinations across k, weighting, and distance metric.

Class	Precision	Recall	F1-Score	Support
Glass	0.63	0.56	0.59	100
Paper	0.74	0.73	0.73	100

Class	Precision	Recall	F1-Score	Support
Cardboard	0.92	0.69	0.79	100
Plastic	0.66	0.69	0.67	100
Metal	0.68	0.54	0.60	100
Trash	0.51	0.78	0.62	100
Overall	0.69	0.67	0.68	600

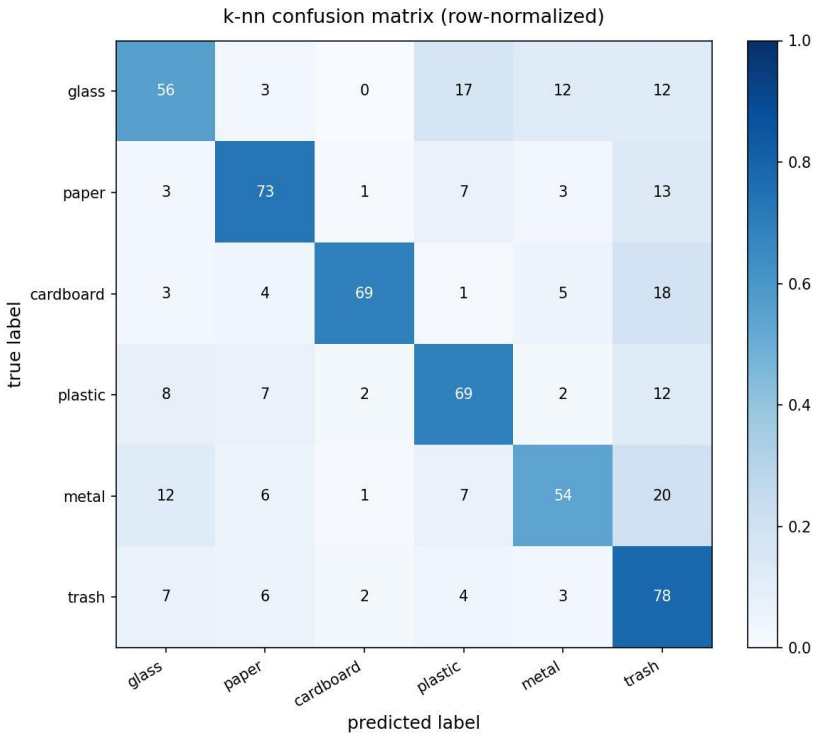

Figure 2: k-NN Confusion Matrix — Validation Set (Accuracy = 66.5%)

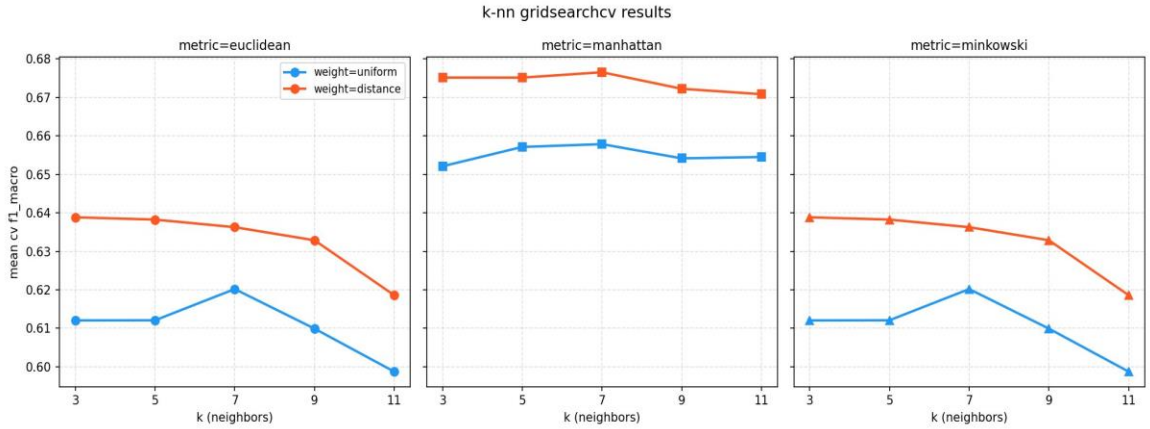

Figure 3: k-NN GridSearchCV Results — Mean CV F1-Macro across k, weighting, and distance metric

5. Results and Comparison

5.1 Accuracy Summary

Model	Best Configuration	Validation Accuracy
SVM	C=5, RBF kernel, gamma=scale	78.50%
k-NN	k=7, manhattan, distance weighting	66.50%

SVM outperforms k-NN by a significant margin of **12%**. **SVM was selected as the deployment model** for the real-time application.

5.2 SVM vs k-NN — Detailed Comparison

Criteria	SVM	k-NN
Validation Accuracy	78.50%	66.50%
Best Configuration	C=5, RBF kernel	k=7, manhattan, distance
Training Time	Slower	Faster
Prediction Time	Faster	Slower
High-Dimensional Data	Handles well	Struggles (curse of dimensionality)
Interpretability	Low	High
Overfitting Risk	Low	High (especially at small k)

5.3 Why SVM Outperforms k-NN

The feature vectors in this system are 738-dimensional. In high-dimensional spaces, distances between points become increasingly uniform — a well-known phenomenon called the curse of dimensionality. This directly hurts k-NN since its entire mechanism relies on meaningful distance measurements between points. Switching to Manhattan distance mitigated this partially, but SVM remains fundamentally better suited to this dimensionality.

SVM is fundamentally different — it finds an optimal separating hyperplane in the feature space and is not affected by this problem in the same way. The RBF kernel further allows it to learn non-linear decision boundaries between material classes, which is necessary since materials like glass and plastic can have overlapping feature distributions.

5.4 Per-Class Comparison

Class	SVM F1	k-NN F1
Glass	0.76	0.59
Paper	0.85	0.73
Cardboard	0.86	0.79
Plastic	0.81	0.67
Metal	0.73	0.60
Trash	0.72	0.62

SVM consistently outperforms k-NN across all six classes. The gap is largest for plastic (0.81 vs 0.67) and metal (0.73 vs 0.60). Both classifiers struggled most with glass, which has low saturation and reflective surfaces that overlap with metal characteristics.

5.5 Limitations and Future Improvements

The current accuracy of 78.5% reflects meaningful progress with handcrafted feature descriptors, though further improvement is possible. Several directions could push performance higher:

- Deep feature extraction — replacing handcrafted descriptors with features from a pretrained CNN would produce richer, more discriminative representations, especially for visually similar classes like glass and metal.
- Larger dataset — the current dataset of 500 images per class is relatively small; more diverse training samples would improve generalization.
- Fine-tuned augmentation — more targeted augmentation strategies per class could help with underperforming classes like glass and metal.

6. System Deployment

6.1 Overview

The best performing model (SVM) was integrated into a real-time camera application that captures live video frames, classifies the material in view, and displays the result on screen. The application is implemented in src/realtime_app.py and runs on a standard webcam without any additional hardware.

6.2 Pipeline

Each frame captured from the camera goes through the following pipeline before a result is displayed:

Step	Operation
1	Capture frame from webcam
2	Extract features using 8 descriptors (HOG, color histograms, color moments, LBP, GLCM, Hu moments, edge features, Gabor) → 738 dimensions
3	Apply StandardScaler (fitted during training)
4	Run SVM — get confidence scores per class
5	Apply rejection mechanism (Unknown if max confidence < 0.5)
6	Display class name and confidence % on the frame

6.3 Rejection Mechanism

The application includes a confidence-based rejection mechanism to handle out-of-distribution inputs — objects that do not belong to any of the six material classes, or inputs that are too ambiguous to classify reliably such as blurred frames or mixed materials.

Since the SVM was trained with **probability=True**, it outputs a confidence score for each of the six classes. If the highest confidence score is below **0.5**, the input is classified as **Unknown**. Otherwise it is classified as the corresponding material. The threshold can be adjusted via the **CONFIDENCE_THRESHOLD** variable in **realtime_app.py**.

6.4 Display

The classification result is overlaid directly on the live camera feed. Each material class is assigned a distinct color for quick visual identification. The result shows the predicted class name and the model's confidence percentage in real time.

Class	Display Color
Glass	Yellow
Paper	Green
Cardboard	Orange
Plastic	Magenta
Metal	Grey
Trash	Red
Unknown	Dark Grey

6.5 Running the Application

Requirements: features/scaler.pkl and models/svm_model.pkl must be present. Run the application with:

```
python src/realtime_app.py
```

Press Q to quit the application.